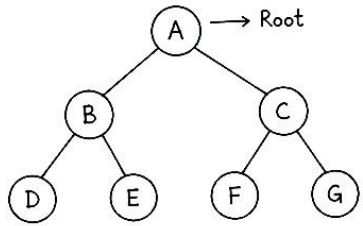
1. a) Define binary tree? Explain the properties of binary tree.

Definition: A binary tree is a finite set of nodes that is either empty or consists of a root node and two disjoint binary trees called the left subtree and the right subtree.



Properties of Binary Tree:

1. Maximum number of nodes at level i (root level = 0) is 2^i.
2. Maximum number of nodes in a binary tree of height h is $2^{h+1} - 1$.
3. For a binary tree with n nodes, maximum number of edges is $n - 1$.
4. For a binary tree with n_0 leaf nodes and n_2 nodes having two children, $n_0 = n_2 + 1$.
5. In a binary tree, for any node, the number of children is either 0, 1 or 2.
6. A binary tree with n nodes has $(n+1)$ NULL (empty) links.

b) Explain the insertion operation on binary search tree with suitable example.

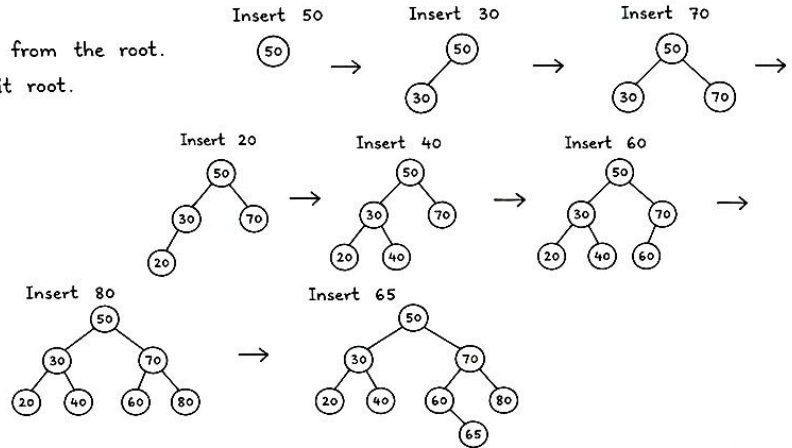
Insertion in BST: To insert a new key in BST, we start from the root.

1. If the tree is empty, create a new node and make it root.
2. Otherwise, compare the key with current node.
   - If key < current node, go to left subtree.
   - If key > current node, go to right subtree.
3. Repeat step 2 until we reach a NULL position.
4. Insert the new node at that position.

> In BST, all keys in left subtree < node key and all keys in right subtree > node key.

Example: Insert 50, 30, 70, 20, 40, 80, 65

Step by step insertion:



2.

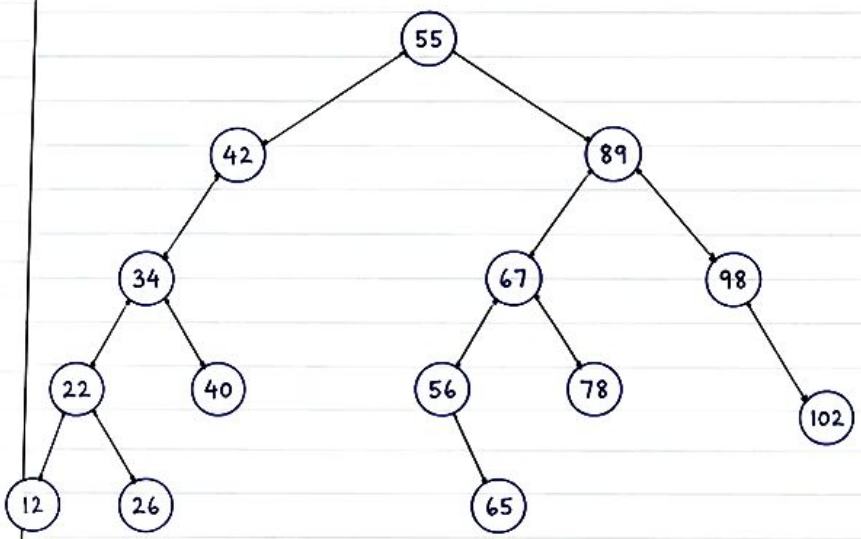
a. Construct the binary search tree for the following keys:

55, 42, 89, 34, 67, 78, 40, 22, 12, 56, 88, 65, 26, 98, 102

Insertion order (step by step):

Step	Key	BST after insertion
1.	55	(root)
2.	42	42 < 55 → left of 55
3.	89	89 > 55 → right of 55
4.	34	34 < 55 → left, 34 < 42 → left of 42
5.	67	67 > 55 → right, 67 < 89 → left of 89
6.	78	78 > 55 → right, 78 < 89 → left, 78 > 67 → right of 67
7.	40	40 < 55 → left, 40 < 42 → left, 40 > 34 → right of 34
8.	22	22 < 55 → left, 22 < 42 → left, 22 < 34 → left of 34
9.	12	12 < 55 → left, 12 < 42 → left, 12 < 34 → left, 12 < 22 → left of 22
10.	56	56 > 55 → right, 56 < 89 → left, 56 < 67 → left of 67
11.	88	88 > 55 → right, 88 < 89 → left of 89
12.	65	65 > 55 → right, 65 < 89 → left, 65 < 67 → left, 65 > 56 → right of 56
13.	26	26 < 55 → left, 26 < 42 → left, 26 < 34 → left, 26 > 22 → right of 22
14.	98	98 > 55 → right, 98 > 89 → right of 89
15.	102	102 > 55 → right, 102 > 89 → right, 102 > 98 → right of 98

Final Binary Search Tree:



b. Construct a Binary tree for the given Inorder and Postorder traversals by explaining the intermediate steps.

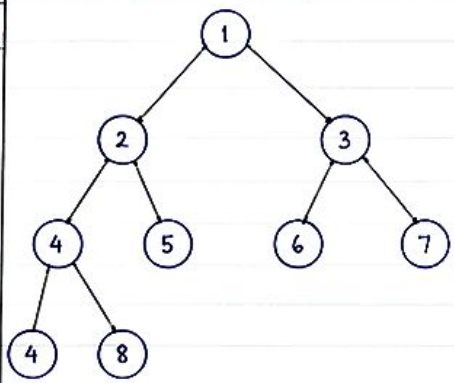
Inorder Traversal : [4, 8, 2, 5, 1, 1, 6, 3, 7]

Postorder Traversal : [8, 4, 5, 2, , 6, 7, 3, 1]

Steps:

Step	Postorder (last is root)	Root	Inorder split (left | root | right)	Left Inorder	Right Inorder
1.	[8, 4, 5, 2, 6, 7, 3, 1]	1	[4, 8, 2, 5] | 1 | [6, 3, 7]	[4, 8, 2, 5]	[6, 3, 7]
2.	[8, 4, 5, 2]	2	[4, 8] | 2 | [5]	[4, 8]	[5]
3.	[8, 4]	4	[4] | 4 | [8]	[4]	[8]
4.	[ ]	4	[4]	–	–
5.	[ ]	8	[8]	–	–
6.	[5]	5	[5]	–	–
7.	[6, 7, 3]	3	[6] | 3 | [7]	[6]	[7]
8.	[6]	6	[6]	–	–
9.	[7]	7	[7]	–	–

Constructed Binary Tree:



Verification:

Inorder of constructed tree : [4, 8, 2, 5, 1, 6, 3, 7] ✓

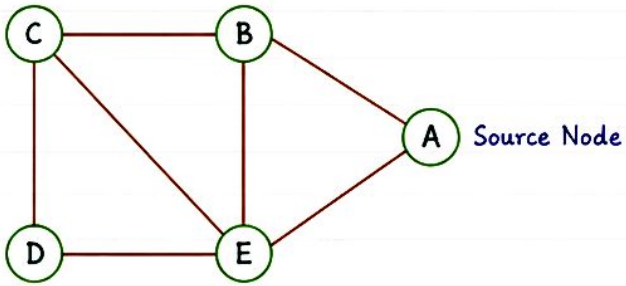
Postorder of constructed tree : [8, 4, 5, 2, 6, 7, 3, 1] ✓

3.

a. **Define Graph?**

A graph G is a non-linear data structure consisting of a finite set of vertices (or nodes) $V = \{v_1, v_2, \ldots, v_n\}$ and a set of edges E that connect pairs of vertices.

It is denoted as $G = (V, E)$.



Example:

Here, $V = \{A, B, C, D, E\}$

$E = \{(C, B), (C, D), (D, E), (B, E), (C, E), (B, A), (E, A)\}$

b. **Which data structure is used in BFS implementation? Explain the step by step procedure to traverse the graph shown in figure 1 using BFS algorithm.**

Data structure used in BFS: **Queue**

BFS Traversal (Starting from Source Node (A)):

Step	Queue (Front → Rear)	Visited	Action
1.	[A]	{A}	Enqueue A (start)
2.	[B, E]	{A, B, E}	Dequeue A; enqueue its unvisited neighbours B, E
3.	[E, C]	{A, B, E, C}	Dequeue B; enqueue its unvisited neighbour C
4.	[C, D]	{A, B, E, C, D}	Dequeue E; enqueue its unvisited neighbour D
5.	[D]	{A, B, E, C, D}	Dequeue C; no unvisited neighbour
6.	[ ]	{A, B, E, C, D}	Dequeue D; no unvisited neighbour

BFS Traversal Order: (A) → (B) → (E) → (C) → (D)

a. Algorithms for DFS and BFS.

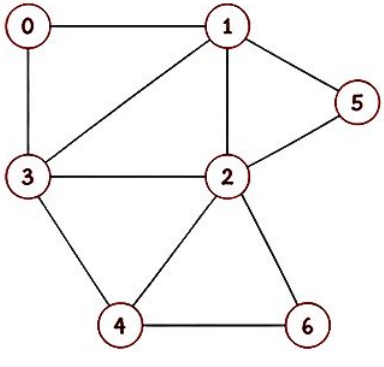
Depth First Search (DFS)

```
Algorithm DFS(G, s)
Input  : Graph G = (V, E), source vertex s
Output : DFS traversal order starting from s
1. for each vertex v in V do
       visited[v] ← false
2. Create an empty stack S.
3. Push s onto stack S.
4. while S is not empty do
       u ← Top(S);  Pop(S)
       if visited[u] = false then
         visit u ;   visited[u] ← true
         For each adjacent vertex v of u
         do if visited[v] = false then
            Push v onto S.
```

Breadth First Search (BFS)

```
Algorithm BFS(G, s)
Input  : Graph G = (V, E), source vertex s
Output : BFS traversal order starting from s
1. for each vertex v in V do
       visited[v] ← false
2. Create an empty queue Q.
3. Enqueue s into Q;   visited[s] ← true
4. while Q is not empty do
       u ← Dequeue(Q)
       visit u
       For each adjacent vertex v of u
       do if visited[v] = false then
          visited[v] ← true
          Enqueue v into Q.
```

b). Trace the algorithms for the following graph.



(Start vertex = 0)

DFS Trace (using Stack)

Step	Stack (top on right)	Visited	Action
1.	[0]	{0}	Visit 0
2.	[3, 1]	{0, 3}	Visit 3
3.	[4, 1]	{0, 3, 4}	Visit 4
4.	[6, 1]	{0, 3, 4, 6}	Visit 6
5.	[1]	{0, 3, 4, 6, 1}	Visit 1
6.	[5, 2]	{0, 3, 4, 6, 1, 5}	Visit 5
7.	[2]	{0, 3, 4, 6, 1, 5, 2}	Visit 2
8.	[ ]	{0, 3, 4, 6, 1, 5, 2}	Done

BFS Trace (using Queue)

Step	Queue (front → rear)	Visited	Action
1.	[0]	{0}	Visit 0
2.	[3, 1]	{0, 3, 1}	Visit 3, 1
3.	[1, 2, 4, 5]	{0, 31, 2, 4, 5}	Visit 2, 4, 5
4.	[2, 4, 5, 6]	{0, 3, 1, 2, 4, 5, 6}	Visit 6
5.	[4, 5, 6]	{0, 3, 1, 2, 4, 5, 6}	–
6.	[5, 6]	{0, 3, 1, 2, 4, 5, 6}	–
7.	[6]	{0, 3, 1, 2, 4, 5, 6}	–
8.	[ ]	{0, 3, 1, 2, 4, 5, 6}	Done

DFS Order : 0 → 3 → 4 → 6 → 1 → 5 → 2

BFS Order : 0 → 3 → 1 → 2 → 4 → 5 → 6